9/03

__/__/8__

Algorithm Merge-Sort (A, low, high)

Input:
A → array
low → starting index
high → ending index

Output:
sorted array A

Step 1: if low < high then
Step 2: mid ← (low + high)/2
Step 3: Merge-Sort (A, low, mid)
Step 4: Merge-Sort (A, mid+1, low)
Step 5: Merge (A, low, mid, high)
Step 6: end if

End Algorithm

Algorithm Merge (A, low, mid, high)

Input:
A → array
low → starting index
mid → middle index

high → ending index

~~Output~~:

Step 1: Create temporary array B
Step 2: $i \leftarrow low$
Step 3: $j \leftarrow mid + 1$
Step 4: $k \leftarrow low$

Step 5: while $i \leq mid$ AND $j \leq high$ do
    if $A[i] \leq A[j]$ then
        $B[k] \leftarrow A[i]$
        $i \leftarrow i+1$
    else
        $B[k] \leftarrow A[j]$
        $j \leftarrow j+1$
    $k \leftarrow k+1$
end while

Step 6: while $i \leq mid$ do
    $B[k] \leftarrow A[i]$
    $i \leftarrow i+1$
    $k \leftarrow k+1$
end while

step 7: while $j \leq high$ do

$B[k] \leftarrow A[j]$

$j \leftarrow j+1$

$k \leftarrow k+1$

end while

step 8: for $k \leftarrow low$ to high do

$A[k] \leftarrow B[k]$

End Algorithm

code for merge sort:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void merge (int A[], int left, int mid, int mid2, int right){
    int i = left;
    int j = mid2;
    int k = 0;
    int temp [60000];
    while ( i <= mid && j <= right) {
        if (A[i] <= A[j]){
            temp [k++] = A[i++];
        else
            temp [k++] = A [j++];
    }
    while (i <= mid)
        temp [k++] = A[i++];
    while (j <= right)
        temp [k++] = A[j++];
```

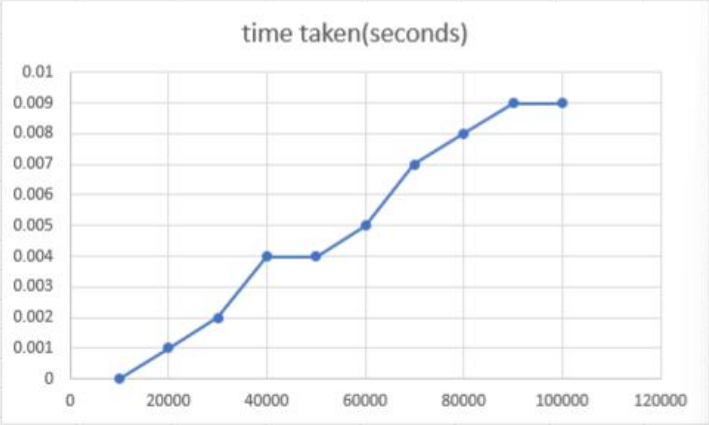
```
    for(i = left, k=0; i<=right; i++, k++){
        A[i] = temp[k];
    }
}

void mergeSort(int A[], int left, int right){
    if (left < right){
        int mid = (left + right)/2;
        mergeSort(A, left, mid);
        mergeSort(A, mid+1, right);
        merge(A, left, mid, mid+1, right);
    }
}

int main(){
    int n, i;
    clock_t start, end;
    double time_taken;
    srand(time(NULL));
    for(n=10000; n<=50000; n+= 10000){
        int A[n];
        for(i=0; i<=n; i++){
            A[i] = rand() % 10000;
        }
        start = clock();
```

```
        mergeSort(A, 0, n-1);
        end = clock();
        time_taken = (double)(end-start)/CLOCKS_PER_SEC;
        printf("n = %d \t Time = %.4f seconds \n", n, time_taken);
    }
    return 0;
}
```

O/p:

n = 10000	Time = 0.0000	seconds
n = 20000	Time = 0.0010	seconds
n = 30000	Time = 0.0030	seconds
n = 40000	Time = 0.0030	seconds
n = 50000	Time = 0.0030	seconds

N	time taken(seconds)
10000	0
20000	0.001
30000	0.002
40000	0.004
50000	0.004
60000	0.005
70000	0.007
80000	0.008
90000	0.009
100000	0.009



"C:\Users\BMSCECSE-L4\Desk

```
N       Time Taken (seconds)
10000   0.000000000000000
20000   0.001000000000000
30000   0.003000000000000
40000   0.003000000000000
50000   0.003000000000000
60000   0.005000000000000
70000   0.006000000000000
80000   0.006000000000000
90000   0.008000000000000
100000  0.009000000000000

Process returned 0 (0x0)   execution time : 0.058 s
Press any key to continue.
```